model.py ↔ 论文 Section Map

baseline/model.py ↔ 论文 Section Map（飞机伴读地图）

> 用法：读完一篇论文的某个 section，立刻翻本文档找到对应代码行号，对照看实现细节。

> 配套：docs/10-research/flight-reading-list-2026-05-06.md

> 打印建议：只打印本文档 + 下列关键代码段（约 600 行），不必打印整个 model.py（2396 行）

顶层地图

model.py 行号	类	对应论文	读后验收问题
11-19	`ModelInput`	— （数据结构）	我输入模型的每个字段 shape 是什么？
26-92	`RotaryEmbedding` + `rotate_half` + `apply_rope_to_tensor`	**论文 2 RoFormer §3.4**	为什么要 rotate_half？怎么把公式(1
100-114	`SwiGLU`	论文辅 + LLaMA 技术报告	为什么用 SiLU 而不是 ReLU/GELU？为什么 gate 后要再一次 Linear？
117-243	`RoPEMultiheadAttention`	**论文 1 Attention §3.2 + 论文 2 §3.4**	Q/K/V reshape 过程？RoPE 在哪一步注入？
244-314	`CrossAttention`	论文 1 + HyFormer Query Decoding	和 self-attention 的唯一区别是什么？
315-414	**`RankMixerBlock`**	**论文 4 RankMixer**	token mixing 的 reshape 过程？per-token FFN 在哪？
415-557	`MultiSeqQueryGenerator`	**论文 3 HyFormer §3.4 Multi-Seq**	每个 domain 为什么独立 FFN？item_conditioned 是什么？
558-598	`SwiGLUEncoder`	— (baseline 的轻量选项)	为什么它比 Transformer 快 3.5×？（attention-free）
600-671	`TransformerEncoder`	论文 1	标准 Pre-LN Transformer，对比 SwiGLUEncoder
672-905	`LongerEncoder`	**论文 5 LONGER**	为什么 cross-attention 省 $O(L^2) \to O(L_H \cdot L_S)$？
906-1043	**`MultiSeqHyFormerBlock`**	**论文 3 HyFormer §3.2-3.4**	三步流程？Q / NS / Seq 的维度怎么变？
1044-1217	**`OneTransMixedAttn`**	**论文 5 OneTrans §Mixed Attn**	Mixed param 参数量？Mixed causal mask 长什么样？
1218-1280	`OneTransMixedFFN`	**论文 5 OneTrans §Mixed FFN**	S 共享 FFN / 每个 NS 独立 FFN 的参数量？
1281-1349	**`OneTransNSConditioner`**	EXP-013 关键差异	为什么它能带来 +0.010 AUC？
1350-1484	`OneTransBlock`	**论文 5 OneTrans**（整体装配）	和 `MultiSeqHyFormerBlock` 的根本差异？
1485-1566	`GroupNSTokenizer`	— (原 baseline 方式)	怎么把 sparse features 聚成 NS tokens？
1567-1688	`RankMixerNSTokenizer`	论文 4 RankMixer 的 NS 部分	当前默认选用，为什么？
1689-END	**`PCVRHyFormer`**	整体装配	数据流从输入到输出完整走一遍

= 飞机上必读

= EXP-013 关键差异源头，必须想清楚

读完"论文 1 Attention Is All You Need"后

立刻翻 model.py:117-243 RoPEMultiheadAttention

关键读法：

- line 141-145：W_q / W_k / W_v / W_o / W_g 五个 Linear（比标准 MHA 多一个 gate W_g，是仓库的自定义增强）
- line 150-200 的 forward：重点看 reshape 过程
  - query: (B, Lq, D) → Q_proj: (B, Lq, D) → Q_heads: (B, h, Lq, d/h)
  - 这是你在论文 1 验收时要画的 reshape 图的真实代码
- line ~200-230：F.scaled_dot_product_attention 是 PyTorch 2.0+ 的融合实现，内部就是 softmax(QK^T/sqrt(d_k))V
- line ~230-243：输出 W_o 投影 + W_g sigmoid gate（论文里没有，是实现增强）

验收问题：

1. 如果我要把这个实现改成不用 RoPE 的标准 MHA，要删哪几行？
2. 如果我想让 K/V 用不同的 num_heads（GQA - Group Query Attention），要改哪里？

立刻翻 model.py:244-314 CrossAttention

关键读法：

- 对比 `RoPEMultiheadAttention`，CrossAttention 的 Q 来源和 K/V 来源不同
- HyFormer Query Decoding 就是用它：Q = query tokens (B, Nq, D)，K/V = sequence tokens (B, L, D)

读完"论文 2 RoPE"后

立刻翻 `model.py:26-92`

三段代码 = 论文三块内容：

代码段	对应论文	读什么
`RotaryEmbedding.__init__` (35-46)	公式 `θ_i = base^(-2i/d)`	`inv_freq = 1/(base^(arange(0,d,2)/d))` 就是 θ_i
`RotaryEmbedding._build_cache` (48-53)	公式 (15) 的预计算	把 `position × θ_i` 的 cos/sin 提前算好存起来
`rotate_half` + `apply_rope_to_tensor` (67-92)	公式 (15) 的应用	**关键 trick**：论文用 block-diagonal 旋转矩阵，代码用 `rotate_ha

深度验收：

- 手动演算 d=4, pos=2 的情况：
  - `inv_freq = [1/base^0, 1/base^(2/4)] = [1.0, 1/sqrt(base)]`
  - `freqs = [2×1.0, 2×1/sqrt(base)] = [2, 2/sqrt(base)]`
  - `emb = cat([freqs, freqs]) = [2, 2/sqrt(base), 2, 2/sqrt(base)]`
  - `cos = [cos(2), cos(2/sqrt(base)), cos(2), cos(2/sqrt(base))]`
  - 设 x = [x0, x1, x2, x3]
  - `rotate_half(x) = [-x2, -x3, x0, x1]`
  - `RoPE(x) = x * cos + rotate_half(x) * sin`
  - 展开：`[x0*cos(2) - x2*sin(2), x1*cos(2/sqrt(base)) - x3*sin(2/sqrt(base)), x2*cos(2) + x0*sin(2), ...]`
- 这和论文公式 (15) 一致吗？（是的，它用的是 "pair-interleave" 简化版本）

读完"论文 3 HyFormer"后

立刻翻 `model.py:906-1043 MultiSeqHyFormerBlock`（核心）

逐段读法：

line 914-966 `__init__`：看它创建了什么

- `self.seq_encoders`：S 个独立编码器（每个 domain 一个）
- `self.cross_attns`：S 个独立 cross-attention（每个 domain 一个）
- `self.mixer = RankMixerBlock(n_total = Nq × S + Nns)`：共享一个 RankMixer

line 968-1043 `forward`：三步对应论文 §3.2-3.4

- Step 1（line ~997-1010）Sequence Evolution：每个序列独立过 encoder

```
for i in range(S):
next_seqs[i], next_masks[i] = self.seq_encoders[i](seq_tokens_list[i], ...)
```

- Step 2（line ~1011-1025）Query Decoding：每个 Q 集合 cross-attend 到对应序列

```
for i in range(S):
q_decoded[i] = self.cross_attns[i](q_tokens_list[i], key=next_seqs[i], value=next_seqs[i])
```

- Step 3（line ~1026-1043）Query Boosting：所有 Q + NS 拼起来过 RankMixer

```
mixed = concat([*q_decoded, ns_tokens], dim=1) # (B, Nq*S + Nns, D)
boosted = self.mixer(mixed)
# 拆回 Q list 和 NS
```

对照论文 Figure 2：飞机上把 Figure 2 放左边，代码放右边，一步一步对。

立刻翻 `model.py:415-557 MultiSeqQueryGenerator`

对应论文 §3.4 Multi-Sequence Extension：

- 每个 sequence 有 N=`num_queries` 个独立 FFN，生成 N 个 Q tokens
- 输入 GlobalInfo_i = Concat(NS tokens mean-pooled, Seq_i mean-pooled)
- 输出 Q_i = [FFN_{i,1}(GlobalInfo_i), ..., FFN_{i,N}(GlobalInfo_i)]

item_conditioned 分支（line 469+）：这是仓库的扩展（DIN-style），论文没有

- 每个 sequence 学一个 gate，混合 "global-info query" 和 "item-aware query"
- 这是你仓库里 `item_conditioned_query` flag 的来源

读完"论文 4 RankMixer"后

立刻翻 `model.py:315-414 RankMixerBlock`（核心）

逐段读法：

line 358-390 `token_mixing` 方法（本类灵魂）：

```
        def token_mixing(self, Q):  # Q: (B, T, D)
            B, T, D = Q.shape
            # Step 1: ■■■■■
            Q = Q.reshape(B, T, T, self.d_sub)   # (B, T, T, d_sub)
            # Step 2: ■■ token ■ subspace ■■
            Q = Q.transpose(1, 2)                # (B, T, T, d_sub)
            # Step 3: flatten ■■
            Q = Q.reshape(B, T, D)               # (B, T, D)
            return Q
```

飞机上手画练习：T=4, d_sub=2, D=8

- 输入 Q[0] = [a0, a1, a2, a3, a4, a5, a6, a7]（一个 token 的 8 维）
- reshape 后 4 个子空间：[a0,a1] [a2,a3] [a4,a5] [a6,a7]
- transpose 后，新的 token 0 channel 来自原 4 个 token 的第 0 个子空间
- 本质 = 沿 token 维度做 fixed permutation，参数为 0

line ~391-414 forward：

- Step 1 token_mixing（参数无关）
- Step 2 Per-token FFN（实际上用的是一个共享 FFN，不是每 token 独立 FFN —— 注意看 `self.fc1` 只有一个 Linear）

仓库 vs 论文的细节差异：

- 论文 RankMixer 原版是"Per-token FFN"：T 个 token 各自有独立 FFN 参数（总参数量 T × FFN size）
- 仓库实现用的是 shared FFN（line 352-353 `self.fc1 = nn.Linear(d_model, d_model * hidden_mult))` → 所有 token 共享一套 FFN 参数
- 这是一个潜在的改进点：如果论文里 Per-token FFN 是核心，而仓库用的 shared，可能你做一个真正的 Per-token FFN 版本就是 +0.002~0.005 AUC 的子弹（需要 proxy 验证）
- 飞机上记下来：T15.x "Per-token FFN 正确实现" 候选

读完"论文 5 OneTrans"后

立刻翻 `model.py:1044-1217 OneTransMixedAttn`（核心）

逐段读法：

line 1058-1092 `__init__`：Mixed Parameterization 在这里体现

- `W_Q_s / W_K_s / W_V_s / W_O_s`（line 1074-1077）：S-tokens 共享一套 $4 \times d^2$ 参数
- `W_Q_ns / W_K_ns / W_V_ns / W_O_ns`（line 1080-1087）：每个 NS token 独立，各 `num_ns` 个 Linear

参数量计算（飞机上记住）：

- Attention 层：`4 × d² × (1 + M)`，M=num_ns。当 M=7 时，比标准 MHA 多 8 倍参数
- FFN 层（`OneTransMixedFFN`）类似

line 1104-1150 `_build_causal_mixed_mask`：一定手画一遍

- 总长度 L+M（L=S-token 数，M=NS-token 数）
- 代码注释说 `S-S ■■■ BIDIRECTIONAL ■■ causal`（因为多 domain 序列按字母序拼的，无时序意义）
- Mask 矩阵四块区域：

```
[ S→S 全 1 | S→NS 全 0 ]
[ NS→S 全 1 | NS→NS causal]
```

- S 看不到 NS（因为 NS 在 S 后面，且代表"目标"，不能前瞻）
- NS 看得到所有 S（NS 是"问题"，需要全部序列上下文）
- NS 内部 causal（保持 NS 之间的顺序依赖）

在纸上手画 L=4, M=3 的 7×7 mask 矩阵（标 T/F），这是 OneTrans 的核心创新。

line 1150-1217 `forward`：

- Step 1 分别投影 S 和 NS 的 Q/K/V（`_project_ns` helper）
- Step 2 拼起来 → 加 mask → scaled_dot_product_attention
- Step 3 分别用 `W_O_s` / `W_O_ns` 做输出投影

立刻翻 `model.py:1281-1349 OneTransNSConditioner`（EXP-013 关键）

为什么它带来 +0.010 AUC？

机制（line 1316-1349）：

```
ns_tokens: (B, M, D)
```

```
compressed_s_list: [(B, K, D)] × S  # ■■ domain ■■■■■■
  ↓
seq_ctx = concat([mean_pool(s) for s in compressed_s_list])  # (B, S*D)
  ↓
for j in range(M):
    delta_j = self.conditioners[j](seq_ctx)  # ■■ NS token ■■■ FFN
    ns_tokens[:, j, :] += delta_j
```

本质：在 OneTrans 主 block 开始前，先让每个 NS token 吸一次序列 mean-pool 信息，避免 NS 从零冷启动去 attend 序列。

为什么这个有效？想清楚（飞机上必答题）：

1. OneTrans v2 无 NS conditioner：NS token 从随机初始化开始 attend 序列，前几层几乎学不到东西（cold start）
2. OneTrans v3 有 NS conditioner：NS 启动时已经带了序列语义，attention 直接聚焦"细节差异"而非"整体含义"
3. 类比 HyFormer 里 `MultiSeqQueryGenerator` 生成 Q tokens 的作用 —— 都是给跨特征交互提供 warm-start
4. 这本身就是一个有"OneTrans 创新"气质的改进 → 可写入 T6 报告，甚至是 Unified Module Innovation Award 的候选素材

读完"论文 6 ESMM"后

没有对应的仓库代码要看（因为赛题数据没有曝光样本，ESMM 路线走不通），但要回顾：

翻 `docs/20-tasks/active.md` 的 T15 段落

- T15 状态：`[x]` 已判定不可行
- 原因：数据全为点击后样本（label_type=1/2），无曝光样本（label_type=0）
- 这是报告里要讲的关键 task framing

翻 `docs/00-context/data-schema.md`

- 看一下 label_type 分布
- 理解本赛题是 click-conditional CVR 而非标准工业 pCVR

飞机落地后的动作清单

按你飞机上的学习深度，回来后的具体动作：

Level 1 - 读懂了论文，能背诵

- [ ] 把 `model.py` 的不懂清单落成 issue / todo
- [ ] 和我同步"5 个问题各自的答案"我帮你验收

Level 2 - 能对照代码，懂设计取舍

- [ ] 启动 T5 OneTrans Pyramid Stack 探索（v3 已有，加 Pyramid）
- [ ] 启动 T-new Per-token FFN 正确实现（RankMixerBlock 当前 shared FFN 可能是漏项）
- [ ] 评估 MoE 分支可行性（论文 4 的 Sparse-MoE 变体）

Level 3 - 能设计创新，有候选 award 角度

- [ ] 起草 Unified Module Innovation Award 申报角度（建议切入点：OneTransNSConditioner 泛化 + Pyramid Stack 组合）
- [ ] 起草 Scaling Law Award 申报角度（建议切入点：复赛 10x 数据下的 d_model / num_blocks / num_queries scaling 曲线）

回来给我反馈的清单

飞机落地请给我：

1. 5 个关键验收问题的答案（每篇论文 1 个，见各论文 section）
2. 你最感兴趣的 2-3 个改进方向（从验收 Level 2/3 的候选里）
3. model.py 里让你还没看懂的段落清单（我带你过）

这样我能帮你把学习转化为实际 leaderboard 提升。